How to get from evaluation to final model

CHRISTOPH MOLNAR
MAR 19, 2024

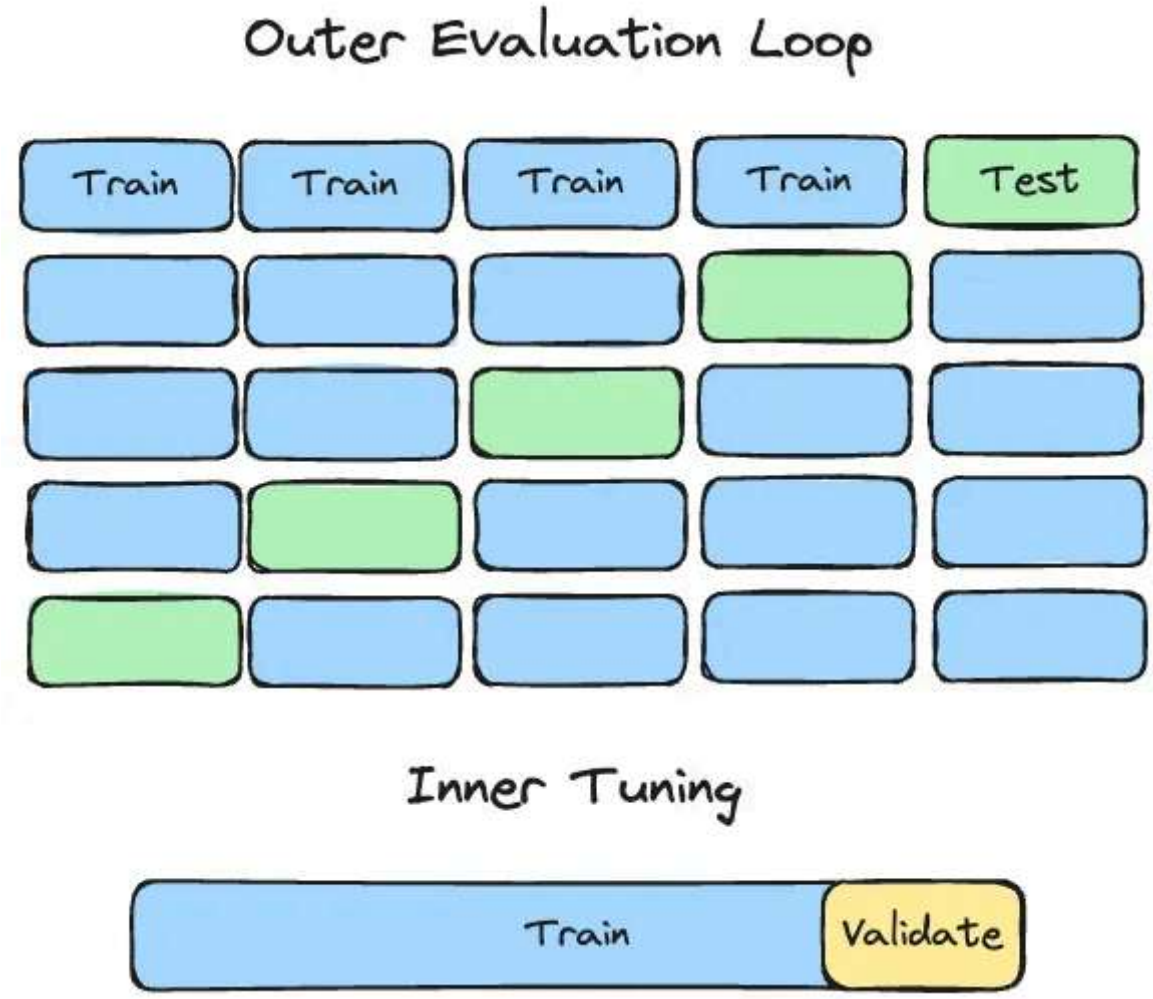
17 5 3 Share

We've all been there: You've set up a machine learning pipeline with tuning, model selection, and evaluation. Tons of data splitting, maybe with cross-validation. You a done modeling and need a final model to deploy.

Except, it's not clear what the "final model" should be, since you have several optio on how to proceed.

Thanks for reading Mindful Modeler! Subscribe for free to receive new posts and support my work.

Let's say you use 5-fold cross-validation to evaluate the model. The model is a gradient-boosted classification model (such as xgboost or Catboost). In each cross-validation iteration, the training data is further split into 80% for training and 20% validation, which is used for tuning hyperparameters with random search. For mor efficient data use you could alternatively tune with an inner cross-validation, but le keep it simple for this example.



Example data splitting setup for training a machine learning model.

Let's say that you get 84% accuracy, which is estimated by averaging the performan over the 5 folds. 84% is good enough to deploy the model. But the pragmatic questi is: What exactly do you deploy now?

And there's a more philosophical side to the question: What does the 84% accuracy refer to? Ideally, whatever model we deploy in the end, we want it to be (at least) 84 accurate, but we also want to make good use of all of our data to make the model m robust and maybe even more accurate.

Let's go through the options:

The Inside-Out Approach: Repeat what you did inside the cross-validation, but no use the entire data. In the example, we have split the data inside the CV into 80%

training and 20% validation for tuning. Then now we take all of the data and train t model with the 80/20 split. The model with the best hyperparameters (highest accuracy) gets deployed. However, this new measure of accuracy is no longer unbias because it's based on the same data we used to tune the model. This approach is consistent with saying that the 84% accuracy applies to the entire model tuning process. Problems with this approach: The final model may have higher variance as depends on a single training and validation split. We didn't use all the data for trair the model, which is another drawback. Also, we have increased the data size, so it's unclear whether the 84% still holds, but typically, it should be better.

The Parameter Donation Approach: Use all of the available data to train the final model. For hyperparameters, use the best hyperparameter configuration from withi the cross-validation. Ideally, the random search used the same random seed in all fc so that each hyperparameter configuration is evaluated across all 5 folds. While this approach allows you to train the model with the most data, there's a conceptual problem: the hyperparameter selection is based on training with only 64% of the overall data (0.8 x 0.8). Hyperparameters are about regularizing the model but the amount of regularization needed depends on the size of the training data. It's uncle what happens with the performance and it might even get worse than the 84%.

The Ensemble Approach: The 5 models from cross-validation are already your fina model. To predict a new data point, simply take the predictions from the five model and average them. This approach is consistent with saying that the evaluation was specific to these five models. Problems with this approach: Makes deployment mor difficult as you now have to deploy an ensemble. It's also unclear whether the accur is still 84% after averaging the predictions for new data, but I would expect the ensemble performance to be equal to or better than for a single model.

There are even more approaches:

- **Lazy approach**: just deploy one of the 5 models from cross-validation.

- **A combination of inside-out and parameter donation**: First you follow the ins out approach and retrain the model using the training/validation split that you used within CV before. Then you take the best hyperparameter configuration a train a model using all data for training.
- **The Fuck-It Approach**: Ignore all previous results. Train a random forest using data. No evaluation. Deploy it. Call it a day.

None of these approaches is perfect, and each has its own trade-offs. It gets even m annoying with more complicated splitting setups like repeated nested cross-validat or conformal prediction.

So unfortunately I don't have a clear answer on the best way to go from modeling tc the final model.

What's your approach to getting from evaluation to final model?

Thanks for reading Mindful Modeler! Subscribe for free to receive new posts and support my work.

17 Likes · 3 Restacks

Discussion about this post

Comments Restacks

Write a comment...